files or artifacts is sent to an external service. That makes it a good fit for more regulated environments or air-gapped scenarios.

```
$ grype .
  ✔ Vulnerability DB        [updated]
  ✔ Indexed .
  ✔ Cataloged packages      [35 packages]
  ✔ Scanned image           [0 vulnerabilities]
No vulnerabilities found
```

NOTE Remember that security is not a static property of a system. At the time of writing, the dependencies used by Catalog Service have no known vulnerabilities, but that doesn't mean this will be true forever. You should scan your projects continuously and apply security patches as soon as they are released, to fix newly discovered vulnerabilities.

Chapters 6 and 7 will cover the remaining steps in the commit stage. For now, let's see how we can automate a deployment pipeline with GitHub Actions. That's the subject of the following section.

3.5.2 Implementing the commit stage with GitHub Actions

When it comes to automating deployment pipelines, there are many solutions from which you can choose. In this book, I'm going to use GitHub Actions (https://github.com/features/actions). It's a managed solution, it provides all the features we need for our project, and it's conveniently already configured for all GitHub repositories. I'm introducing this topic early in this book so that you can use the deployment pipeline to verify your changes while working on the project throughout the book.